

06 — Plano de Expansão: Kit do Consultor e Kit Distribuidor

Status: plano aprovado, aguardando implementação · **Escopo:** 2 novos tipos de material (kit-consultor, kit-distribuidor), cada um com 10 peças (copy + arte 1080×1350 + texto de WhatsApp), organizadas em 5 categorias de tom.

1. Objetivo

Adicionar à fábrica dois novos materiais selecionáveis no /esbocar, com predefinições **fixas** (não interview-dependentes, exceto pela seleção sim/não em materiais_selecionados):

Predefinição	Valor fixo
Público-alvo	Dentista/Implantodontista (novo, distinto de consultores/clientes/distribuidores do projeto)
Produto	O produto foco do projeto atual (mesma imagem/fatos já em <code>dossie_insumos.md</code>)
Tons de voz	5: Informativo, Contra-intuitivo, Técnico, Efeito “uau”, Educativo
Formato de arte	Único: 1080×1350
Modelo de renderização	O já estabelecido para arte (HTML/CSS + Playwright, <code>template arte-1080x1350.html</code> , design system fixo)

Cada kit contém **10 itens** (5 tons × 2 variações por tom), cada item = 1 copy + 1 PNG 1080×1350 + 1 texto de WhatsApp.

2. Decisão de arquitetura confirmada com o operador

Kit do Consultor e Kit Distribuidor compartilham o mesmo conteúdo-base. As 10 copies (headline + subcopy + tom + ângulo) são escritas **uma única vez** por projeto. O que distingue um kit do outro é **apenas o CTA final e a assinatura de rodapé** (ex.: “Fale com seu consultor Conexão” vs. “Peça ao seu distribuidor Conexão”), aplicados de forma determinística (sem 2ª chamada de LLM) no momento da renderização.

Isso é a mesma lógica arquitetural já usada para `arte-01/02/03` (`arte/copies.json` compartilhado, formato como eixo ortogonal) — aqui o eixo ortogonal é **variante de kit** (consultor/distribuidor) em vez de formato, já que o formato dos kits é único (1080×1350).

3. Estrutura de diretórios (por projeto)

```
output/<slug>/
├── kits/
│   └── copies.json                ← 10 copies compartilhadas (fonte única)
├── kit-consultor/
│   ├── artes-informativas/
│   │   └── arte-01/ {conteudo.json, index.html, arte_<slug>_kit-
```

consultor_informativa_01.png, texto_whatsapp.txt}

```
| | | arte-02/ {...}
| | | arte-contraintuitivas/
| | | | arte-01/ {...}
| | | | arte-02/ {...}
| | | artes-tecnicas/
| | | | arte-01/ {...}
| | | | arte-02/ {...}
| | | artes-efeito-uau/
| | | | arte-01/ {...}
| | | | arte-02/ {...}
| | | artes-educativas/
| | | | arte-01/ {...}
| | | | arte-02/ {...}
| | kit-distribuidor/
```

| | (estrutura idêntica a kit-consultor/ – mesmas 10 copies, CTA/assinatura diferente)

kits/ é uma pasta auxiliar (mesmo padrão de insumos/, revisao/, arte/) – não é um material final por si, é o artefato compartilhado que os 2 kits consomem.

Nomenclatura de arquivo

arte_<slug>_<variante-kit>_<tom>_<NN>.png, ex.: arte_kit-inlego_kit-consultor_informativa_01.png. Os três eixos (kit, tom, item) aparecem sempre juntos – mesma disciplina de nunca deixar um número solto que SPEC_ARTE.md já estabelece para arte_<formato>_copy<NN>.

4. Novos artefatos de referência fixos

4.1 brand/tons-kit.json (novo, fixo – mesmo papel de publicos-alvo.json)

Define os 5 tons com descrição + exemplo, para que redator-kit-copy produza ângulos genuinamente distintos (não 5 variações do mesmo tom):

Tom	Definição	Exemplo de headline
informativa	Apresenta um fato/dado direto sobre o produto ou o problema, sem tentar surpreender ou ensinar processo.	“Kit inLego é compatível com sistemas MU e MB.”
contra-intuitiva	Desafia uma crença comum do dentista/implantodontista — quebra expectativa.	“Mais peças no kit não significa mais complexidade na cadeira.”
tecnica	Foco em especificação/mecanismo técnico preciso (material, dimensão, torque, compatibilidade).	“PEEK Grau Médico, sem parafuso passante, rosqueamento manual horário.”
efeito-uau	Um resultado/número surpreendente com impacto imediato.	“Metade do tempo de escaneamento. Um clique, dois problemas resolvidos.”
educativa	Ensina um conceito ou explica o “porquê” de uma prática clínica, tom didático.	“Por que a mandíbula é o ‘deserto anatômico’ do escaneamento digital?”

4.2 brand/publicos-alvo.json (estendido)

Nova entrada dentista_implantodontista — **kit-only, não selecionável nas rodadas 2/3 do /esbocar** (o publico_alvo do projeto continua sendo consultores/clientes/distribuidores; os kits têm o próprio público fixo, independente da escolha do operador para o resto do projeto).

4.3 brand/kits-conexao.json (novo, fixo)

Config determinística da única diferença entre os 2 kits — sem LLM:

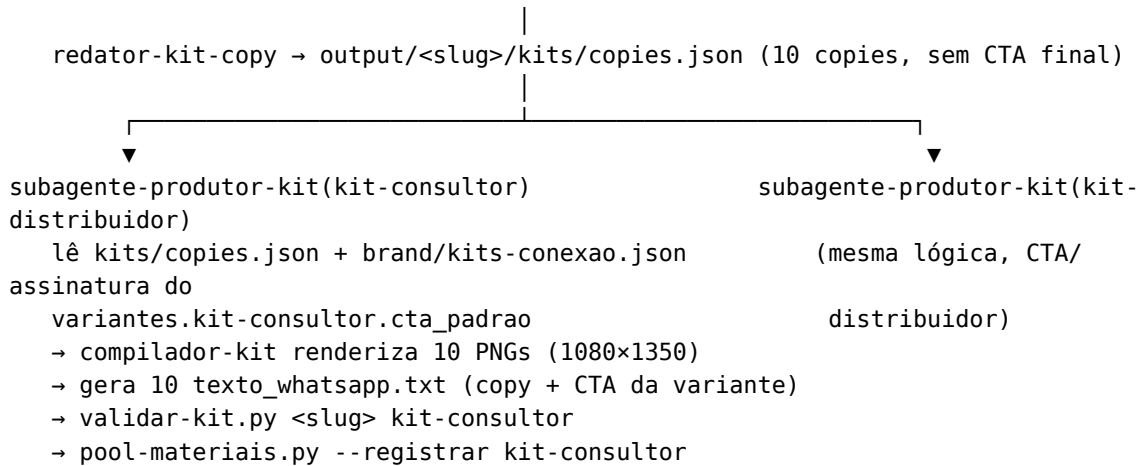
```
{
  "publico_fixo": "dentista_implantodontista",
  "formato_fixo": "1080x1350",
  "tons": ["informativa", "contra-intuitiva", "tecnica", "efeito-uau",
"educativa"],
  "variantes": {
    "kit-consultor": {"cta_padrao": "Fale com seu consultor Conexão",
"assinatura": "Consultor Conexão"},
    "kit-distribuidor": {"cta_padrao": "Peça ao seu distribuidor Conexão",
"assinatura": "Distribuidor Autorizado Conexão"}
  }
}
```

5. Pipeline

diretor-de-arte (Fase 1, já roda no /esbocar)

↳ brief_criativo.mapeamento_por_material.kit.angulos_por_tom

(2 ângulos por tom × 5 tons = 10 ângulos-base, SEM CTA – kit-agnóstico)
 |
 /produzir-comunicacao-completa – Passo 2.7 (NOVO, mesma disciplina do Passo 2.5 de arte)
 Se kit-consultor OU kit-distribuidor selecionado e output/<slug>/kits/copies.json
 ainda não existir: invoca redator-kit-copy UMA ÚNICA VEZ (nunca por subagente de kit)



compilador-kit reaproveita a técnica de compilador-arte (Playwright + template arte-1080x1350.html + design system fixo), fatorando a lógica comum de template/asset em um helper compartilhado (scripts/_arte_common.py) para não duplicar a técnica entre compilar-arte.py e o novo compilar-kit.py — os modelos de dados são diferentes o bastante (1 copy compartilhada × 3 formatos vs. 10 copies × 1 formato × 2 variantes de CTA) para não caber na mesma função sem condicionais confusas.

6. Granularidade de subagente

1 subagente-produtor-kit por **kit inteiro** (kit-consultor ou kit-distribuidor), não por tom — mesma convenção já usada para pdf/landing-page/apresentacao/textos (1 subagente = 1 entrada de materiais_selecionados). Cada subagente produz os 10 itens do seu kit internamente, em loop, chamando compilador-kit uma vez por item.

pool-materiais.py trata kit-consultor e kit-distribuidor como 2 materiais de topo adicionais — material_entregue() exige exatamente 10 PNGs + 10 conteudo.json + 10 texto_whatsapp.txt presentes (mesma disciplina de contagem exata já usada para arte-0N com as 3 copies).

7. Validação: novo scripts/validar-kit.py

```
python scripts/validar-kit.py <slug> kit-consultor
python scripts/validar-kit.py <slug> kit-distribuidor
```

Critérios (exit 1 se qualquer falhar): - As 5 pastas de tom existem (artes-informativas, artes-contra-intuitivas, artes-tecnicas, artes-efeito-uau, artes-educativas). - Cada uma tem exatamente 2 subpastas arte-01/arte-02. - Cada arte-0N tem exatamente 1 PNG

1080×1350 (pixel-perfect, < 1 MB — mesmo teto de SPEC_ARTE.md), 1 conteudo.json não vazio, 1 texto_whatsapp.txt não vazio em UTF-8. - Total: 10 PNGs + 10 conteudo.json + 10 texto_whatsapp.txt por kit.

8. Arquivos novos

Arquivo	Papel
SPEC_KITS.md	Contrato técnico completo (estrutura, nomenclatura, predefinições, validação) — sibling de SPEC_ARTE.md/SPEC_PDF.md/SPEC_HTML.md.
brand/tons-kit.json	Definição fixa dos 5 tons (seção 4.1).
brand/kits-conexao.json	CTA/assinatura fixos por variante de kit (seção 4.3).
.claude/skills/redator-kit-copy/SKILL.md	Gera as 10 copies compartilhadas (uma vez por projeto).
.claude/skills/compilador-kit/SKILL.md	Renderiza os 20 PNGs finais (10 por kit) + os 20 texto_whatsapp.txt.
.claude/agents/subagente-produtor-kit.md	1 subagente por kit (kit-consultor/kit-distribuidor), model: inherit (REGRA 5).
scripts/compilar-kit.py	Compilador determinístico (Playwright), lê kits/copies.json + brand/kits-conexao.json.
scripts/validar-kit.py	Validação estrutural (seção 7).
scripts/_arte_common.py	Helper compartilhado extraído de compilar-arte.py (template substitution + cópia de assets + render Playwright), reaproveitado por compilar-kit.py.

9. Arquivos alterados

Arquivo	Mudança
.claude/commands/esbocar.md	Passo 4 ganha 2 novas opções (“Kit do Consultor”, “Kit Distribuidor”); com 9 opções totais, a pergunta multiSelect precisa ser dividida em 3 partes (máx. 4 opções por AskUserQuestion), não mais 2.
.claude/commands/produzir-comunicacao-completa.md	Novo Passo 2.7 (copy de kit compartilhada, mesma disciplina do Passo 2.5 de arte) + dispatch de subagente-produtor-kit no fan-out.
.claude/skills/diretor-de-arte/SKILL.md	mapeamento_por_material.kit — 10 ângulos-base (2 por tom), público fixo dentista_implantodontista, sem CTA (kit-agnóstico).
.claude/skills/revisor-marca/SKILL.md	Critério para kit-consultor/kit-distribuidor: rodar validar-kit.py; confirmar que as 10 copies são idênticas entre os 2 kits exceto CTA/assinatura; confirmar 5 ângulos de tom genuinamente distintos.
scripts/compilar-arte.py	Refatorado para reaproveitar scripts/_arte_common.py (sem mudança de comportamento externo).
scripts/pool-materiais.py	TITULOS_MATERIAL + material_entregue() para kit-consultor/kit-distribuidor (10 PNGs/conteudo/textos exatos).
scripts/empacotar-projeto.py	PATH_POR_TIPO com resolver para os 2 novos tipos (retorna a pasta do kit se os 30 arquivos esperados — 10+10+10 — existirem).
scripts/auditar-projeto.py	TIPOS_VALIDOS + rodar_validador() (chama validar-kit.py) + R12 (novas pastas esperadas).
scripts/parametros_projeto.py	TIPOS_VALIDOS (linha 28) — enum próprio, separado do de auditar-projeto.py , precisa das 2 novas entradas ou config_projeto.json com kit selecionado falha na validação do Passo 5 do /esbocar.
scripts/verificar-consistencia-pipeline.py	TIPOS_VALIDOS + MAPA_ESBOCAR/MAPA_DISPATCH/MAPA_SKILL/MAPA_AGENTE/MAPA_VALIDADOR — sem isso, o guarda-corpo
scripts/preferencias-compatibilidade-slug.py	7 uma nova classe de lacuna (material funcional em algumas camadas mas ausente em outras) não vai saber que kits existem uma nova classe de lacuna (material funcional em algumas camadas mas ausente em outras) não vai saber que kits existem

10. Hooks / MCP (code-review-graph)

Nenhuma mudança de configuração é necessária. Os novos scripts (`compilar-kit.py`, `validar-kit.py`, `_arte_common.py`) são automaticamente descobertos e indexados pelo hook `PostToolUse/SessionStart` já configurado — a única ação recomendada após a implementação é rodar `run_postprocess_tool` (ou aguardar o próximo `SessionStart`) para que o grafo reflita os novos módulos antes da próxima sessão de revisão de código (REGRA 9).

11. Sequência de implementação recomendada

1. Artefatos de referência fixos: `brand/tons-kit.json`, `brand/kits-conexao.json`, extensão de `brand/publicos-alvo.json`.
2. `SPEC_KITS.md` (contrato completo, antes de qualquer código — mesma ordem já seguida para `SPEC_ARTE.md`).
3. Refatorar `compilar-arte.py` → extrair `scripts/_arte_common.py` (com regressão validada em `kit-inlego` antes de prosseguir — não pode quebrar o pipeline de arte existente).
4. `scripts/compilar-kit.py` + `scripts/validar-kit.py` (usando o helper comum).
5. Skills: `redator-kit-copy`, `compilador-kit`; agente `subagente-produtor-kit.md`.
6. Atualizar `diretor-de-arte`, `revisor-marca`.
7. Atualizar orquestração: `esboçar.md` (Passo 4), `produzir-comunicacao-completa.md` (Passo 2.7 + `dispatch`).
8. Atualizar guarda-corpos: `parametros_projeto.py`, `verificar-consistencia-pipeline.py`, `auditar-projeto.py`, `empacotar-projeto.py`, `pool-materiais.py`, `preflight-compatibilidade-slug.py`.
9. Atualizar `AGENTS.md/SPEC.md`.
10. Rodar `verificar-consistencia-pipeline.py --estrito` — deve retornar OK antes de qualquer produção real.
11. Teste ponta a ponta: adicionar `kit-consultor` + `kit-distribuidor` a `output/kit-inlego/config_projeto.json` (ou rodar `/gerar-kit kit-inlego`, se um comando pontual equivalente a `/gerar-arte` for criado) e produzir os 2 kits completos, com `auditar-projeto.py --estrito` CONFORME ao final.

12. Riscos e mitigação

- **Risco:** confundir o eixo “tom” (5, kit) com o eixo “objetivo_tom” (3, projeto) — são vocabulários parecidos mas independentes. **Mitigação:** nomear sempre com o prefixo `tom_kit/brand/tons-kit.json` na documentação e no código, nunca reaproveitar o campo `tom_de_voz` do brief para os kits.
- **Risco:** duplicar 100% da lógica de `compilar-arte.py` em `compilar-kit.py`, criando 2 pontos de manutenção para o mesmo bug (ex.: o bug de path relativo já corrigido 2 vezes nesta fábrica). **Mitigação:** helper compartilhado `_arte_common.py` (seção 5).
- **Risco:** achar que só precisa atualizar `pool-materiais.py/auditar-projeto.py` e esquecer `parametros_projeto.py` (que tem seu **próprio** `TIPOS_VALIDOS`, redundante e não sincronizado automaticamente) — isso já quase causou o bug histórico do material “textos” citado no docstring de `verificar-consistencia-pipeline.py`. **Mitigação:** rodar esse script `--estrito` como gate antes de declarar a expansão pronta (passo 10 da seção 11).

- **Risco:** achar que kit-consultor e kit-distribuidor precisam de 20 copies independentes (custo de LLM em dobro) — já resolvido pela decisão da seção 2.

13. Critérios de aceite

- SPEC_KITS.md existe e cobre 100% da estrutura desta seção.
- verificar-consistencia-pipeline.py --estrito retorna OK incluindo kit-consultor/ kit-distribuidor em todas as 5 camadas (entrevista, dispatch, skill, agente, validador).
- Rodando /produzir-comunicacao-completa kit-inlego com os 2 kits selecionados: output/kit-inlego/kits/copies.json tem exatamente 10 copies; kit-consultor/ e kit-distribuidor/ têm cada um exatamente 10 PNGs 1080×1350 + 10 conteudo.json + 10 texto_whatsapp.txt; as copies são idênticas entre os 2 kits exceto CTA/assinatura.
- auditar-projeto.py kit-inlego --estrito → CONFORME incluindo os 2 novos materiais.
- manifesto_materiais.json referencia kit-consultor/ e kit-distribuidor/ com os 30 arquivos cada (10+10+10).